Architectural Styles

Lecturer: Ngo Huy Bien
Software Engineering Department
Faculty of Information Technology
VNUHCM - University of Science
Ho Chi Minh City, Vietnam
nhbien@fit.hcmus.edu.vn

Objectives

- To apply an *architectural style* in creating architecture.

Contents

References

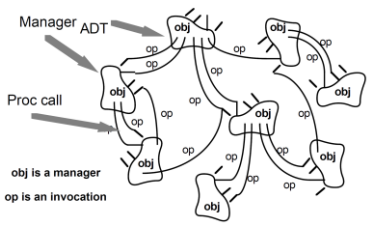
1. David Garlan and Mary Shaw (1993). An Introduction to Software Architecture.
2. Frank Buschmann et al. (1996) Pattern Oriented Software Architecture A System Of Patterns. John Wiley & Sons Ltd.
3. Craig Larman (2004). Applying UML And Patterns. 3rd Edition. Prentice Hall.
4. Microsoft Corporation (2003). Enterprise Solution Patterns Using Microsoft .NET. Microsoft Press.
5. Gautam Shroff (2010). Enterprise Cloud Computing: Technology Architecture Applications.

Architectural Styles [1]

- An *architectural style* defines a family of systems in terms of a pattern of structural organization which includes:
  - the *vocabulary* of components and connectors that can be used in instances of that style
  - a *set of constraints* on how they can be combined. For example, one might constrain the topology of the descriptions (e.g., no cycles) and execution semantics (e.g., processes execute in parallel), and
  - an *informal description* of the benefits and drawbacks of using that style.

Data Abstraction and Object-Oriented Organization (I)

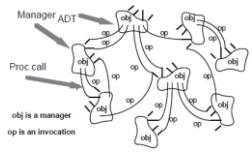
- What does the style look like?
- How to use the style?
  - Language support.
- Why use the style?



Abstract Data Types and Objects

Data Abstraction and Object-Oriented Organization (II)

- In *data abstraction and object-oriented organization style* data representations and their associated primitive operations are encapsulated in an abstract data type or object.
- The components of this style are the objects — or, if you will, instances of the abstract data types.
- Objects interact through function and procedure invocations.



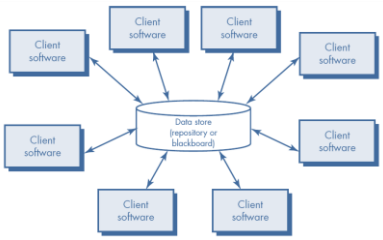
Data Abstraction and Object-Oriented Organization (III)

- Pros and
  - Allows designers to decompose problems into collections of interacting agents.
  - It is possible to *change the implementation* without affecting those clients
- Cons
  - In order for one object to interact with another (via procedure call) it must *know the identity* of that other object.
  - This is in contrast, for example, to pipe and filter systems, where filters do need not know what other filters are in the system in order to interact with them.
  - The significance of this is that whenever the identity of an object changes it is necessary to modify all other objects that explicitly invoke it.

Main Program and Subroutines

- What does the style look like?
- How to use the style?
  - Language support.
- Why use the style?

Repository Style



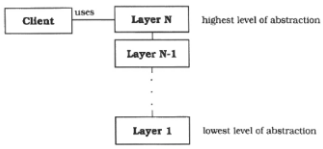
Data Lake

Layers: *Problem* [2]

- Context
  - A *large system* that requires decomposition.
- Forces:
  - Late *source code changes* should not ripple through the system. They should be confined to one component and not affect others.
  - Parts of the system should be *exchangeable*. Components should be able to be replaced by alternative implementations without affecting the rest of the system.
  - Similar responsibilities should be grouped to help *understandability* and *maintainability*.
  - The system will be built by *a team of programmers*, and work has to be *subdivided* along clear boundaries.

Layers: *Solution*

- Solution:
  - Structure your system into an appropriate number of layers and place them on top of each other.
- Structure:

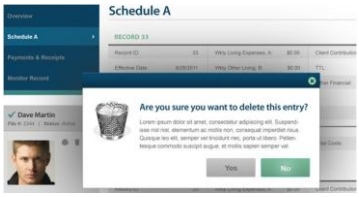


What is a Layer? [3]

A *layer* is a very coarse-grained grouping of classes, packages, or subsystems that has cohesive responsibility for a major aspect of the system.

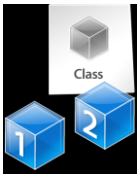
Presentation Layer [4]

- *UI components* – forms, dialogs, controls.
- *Presentation logic components* – encapsulates dependencies, validation, and navigation between components.



Business Layer

- Implements the *business functionality* of the application.
- *Business entities* – data containers. Example: DTO.
- *Business components* – encapsulates the business logic. Example: business rules.
- *Business workflows* – encapsulates the business processes. Example: Order processing flow, customer support flow.
- *Service interfaces* – presents this service to the outside world.



Data Layer

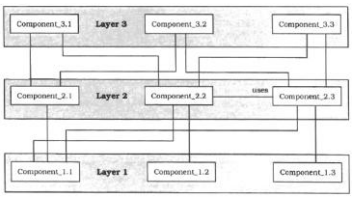
- Provides *access to external systems* such as databases, file systems.
- *Data access components* – exposes the data stored in these databases to the business layer (ADO.NET, Object/Relational mapping tools)
- *Service gateways* – encapsulates the interface, protocol, and code required to access internal and external services or applications.

An Individual Layer [2]

Class Layer J	**Collaborator** • Layer J-1
Responsibility • Provides services used by Layer **J+ 1.** • Delegates subtasks to Layer **J-1.**	

Top-Down Communication

- A client issues a *request* to Layer N.
- Since Layer N *cannot* carry out the request on its own, it calls the next Layer N - 1 for supporting subtasks.
- Layer N - 1 provides these, in the process sending further requests to Layer N-2, and so on *until* Layer 1 is reached.



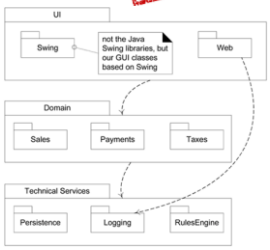
Bottom-Up Communication

- A chain of actions *starts at Layer 1*, for example when a device driver detects input.
- The driver translates the input into an internal format and *reports it to Layer 2* which starts interpreting it, and so on.
- In this way data moves up through the layers *until it arrives at* the highest layer.

- While top-down information and control flow are often described as '*requests*', bottom-up calls can be termed '*notifications*'.

Three-Layered Architecture [4]

- Typically *layers* in an OO system include:
  - User interface.
  - Application logic and domain objects software objects.
  - Technical services.
- In a *strict layered architecture*, a layer only calls upon the services of the layer directly below it.
- This design is common in network protocol stacks, but not in information systems, which usually have a *relaxed layered architecture*, in which a higher layer calls upon several lower layers.



Augmented Techniques [2]

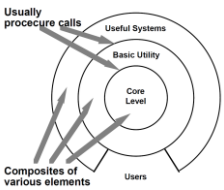
- *Layer Supertype* – Extracts common behaviors into a common class or component from which all the components in the layer inherit.
- *Abstract Interface* – Defines abstract interface for each component in a layer that is called by components in a higher level.
- *Layer Facade* – Provides a single unified interface to a layer or subsystem instead of developing an abstract interface for each exposed component.

Layer Benefits

- *Reuse* functionality exposed by the various layers.
- *Testability* benefits from having well-defined layer interfaces.
- *Deploy work* at layer boundaries.
- Maintenance and enhancement are easier due to *local dependencies*.

Layered Systems [1]

- A *layered system* is organized hierarchically, each layer providing service to the layer above it and serving as a client to the layer below.
- In *some* layered systems inner layers are hidden from all except the adjacent outer layer, except for certain functions carefully selected for export.
- The connectors are defined by the protocols that determine how the layers will interact.
- Topological constraints include limiting interactions to adjacent layers.



Layered System Pros

- They support design based on increasing levels of abstraction. This allows implementers to *partition a complex problem* into a sequence of incremental steps.
- They support enhancement. because each layer interacts with at most the layers below and above, changes to the function of one layer *affect at most two other layers*.
- They support reuse. *Different implementations* of the same layer can be used interchangeably, provided they support the same interfaces to their adjacent layers.

Layered System Cons

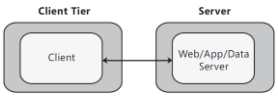
- Not all systems are easily *structured in a layered fashion*.
- Even if a system can logically be structured as layers, *considerations of performance* may require closer coupling between logically high-level functions and their lower-level implementations.
- It can be quite difficult to find the *right levels of abstraction*. The communications community has had some difficulty mapping existing protocols into the ISO framework: many of those protocols bridge several layers.

Single-Tiered Architecture [4]

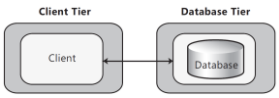
One computer does all the processing.

2-Tier Deployment [5]

- Consider the 2-tier pattern if you are developing *a client* that will access an *application server*, or a stand-alone client that will access a separate *database server*.



A common Web application scenario where the client interacts with a Web server.



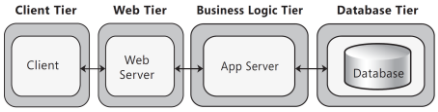
All of the application code is located on the client.
The client makes use of stored procedures or minimal data access functionality on the database server.

3-Tier Deployment

- In a 3-tier design, *the client* interacts with *application software* deployed on a separate server, and the application server interacts with a *database* that is located on another server.
- You might implement a *firewall* between the client and the Web/App tier, and another firewall between the Web/App tier and the database tier.

4-Tier Deployment

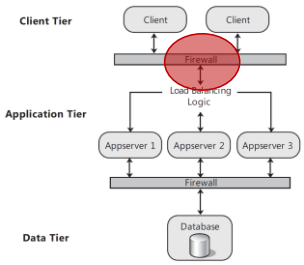
- Consider the 4-tier pattern if *security requirements* dictate that business logic cannot be deployed to the perimeter network, or you have application code that makes heavy use of resources on the server and you want to *offload* that functionality to another server.



Examples: API, email, reporting services.

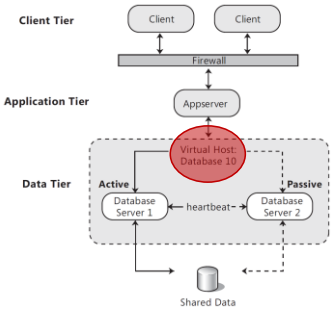
Load-Balanced Cluster

- You can install your service or application onto multiple servers that are configured to *share the workload*.
- Load-balanced clusters are most scalable and efficient if they do not have to track and store information between each client request; in other words, if they are *stateless*.
- If they must track state, then you may need to use *affinity* and *session techniques*.

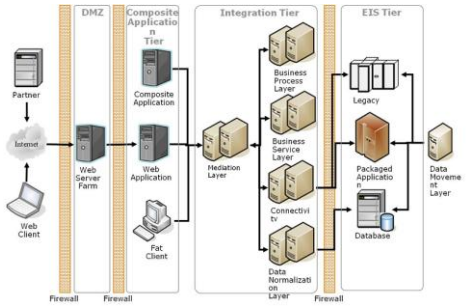


Failover Cluster

- A *failover cluster* is a set of servers that are configured in such a way that if one server becomes unavailable, another server automatically takes over for the failed server and continues processing.



Multi-Tiered Architecture

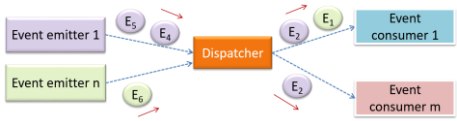


Distributes your application over *multiple servers*.

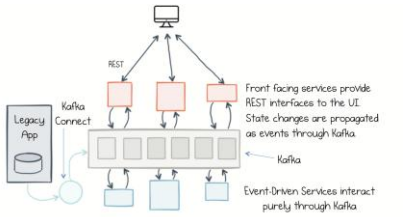
Tier Benefits [4]

- Each tier can be optimized for *specific resource usage* profiles.
- Each tier can have a *different security profile*.
- Each tier can be *separately modified* to respond to changes in load, requirements, hosting strategy, and so on.
- Tiers can be deployed to *meet* geographical, political, and legal requirements.

Event-Based Style: GUI



Event-Based Style: Event Bus



Event-Based, Implicit Invocation (I)

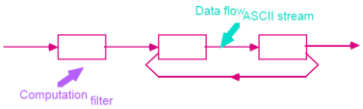
- The idea behind *implicit invocation* is that instead of invoking a procedure directly, a component can announce (or broadcast) one or more events.
- Other components in the system can register an interest in an event by associating a procedure with the event.
- When the event is announced the system itself *invokes* all of the procedures that have been registered for the event.
- Thus an event announcement "implicitly" causes the invocation of procedures in other modules.

Event-Based, Implicit Invocation (II)

- Architecturally speaking, the components in an *implicit invocation style* are modules whose interfaces provide both a collection of procedures (as with abstract data types) and a set of events.
- Procedures may be called in the usual way. But in addition, a component can register some of its procedures with events of the system. This will cause these procedures to be invoked when those events are announced at run time.
- Thus the connectors in an implicit invocation system include traditional procedure call as well as bindings between event announcements and procedure calls.

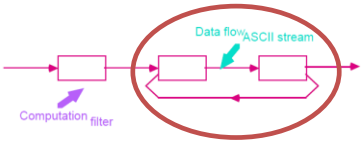
Pipes and Filters (I) [1]

- In a *pipe and filter style* each component has a set of inputs and a set of outputs.
- A *component* reads streams of data on its inputs and produces streams of data on its outputs, delivering a complete instance of the result in a standard order.
- The *connectors* of this style serve as conduits for the streams, transmitting outputs of one filter to inputs of another.



Pipes and Filters (II)

- Constraints
  - Filters must be *independent entities*: in particular, they should not share state with other filters.
  - Filters *do not know the identity* of their upstream and downstream filters.
  - The correctness of the output of a pipe and filter network should not depend on the order in which the filters perform their *incremental processing*.



Pipes and Filters (III)

- Pros
  - Support *reuse*: any two filters can be hooked together, provided they agree on the data that is being transmitted between them.
  - Systems can be easily *maintained and enhanced*: new filters can be added to existing systems and old filters can be replaced by improved ones.
  - Naturally support *concurrent execution*: each filter can be implemented as a separate task and potentially executed in parallel with other filters.
- Cons
  - Handling *interactive* applications: the designer is forced to think of each filter as providing a *complete* transformation of input data to output data.

Example 1

- http://blog.petersobot.com/pipes-and-filters

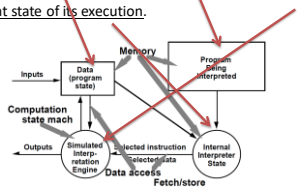
```
cat /usr/share/dict/words |     # Read in the system's dictionary.
grep purple |                   # Find words containing 'purple'
awk '{print length($1), $1}' |  # Count the letters in each word
sort -n |                       # Sort lines ("${length} ${word}")
tail -n 1 |                     # Take the last line of the input
cut -d " " -f 2 |               # Take the second part of each line
cowsay -f tux                   # Put the resulting word into Tux's
mouth
```

Example 2

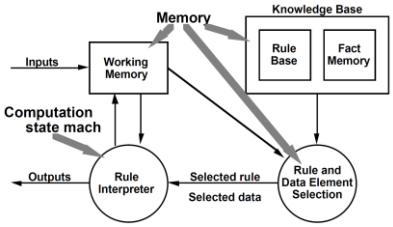
- https://learn.microsoft.com/en-us/azure/architecture/patterns/pipes-and-filters
- Decompose a task that performs complex processing into a series of separate elements that can be reused.
- https://github.com/mspnp/cloud-design-patterns/tree/main/pipes-and-filters

Interpreter Organization

- An *interpreter* includes the pseudo-program being interpreted and the interpretation engine itself.
- The *pseudo-program* includes the program itself and the interpreter's analog of its execution state (activation record).
- The *interpretation engine* includes both the definition of the interpreter and the current state of its execution.

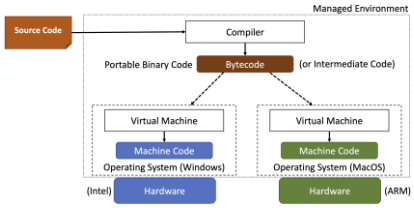


Basic Rule-Based System

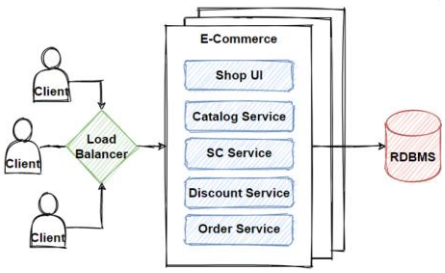


Java Virtual Machine

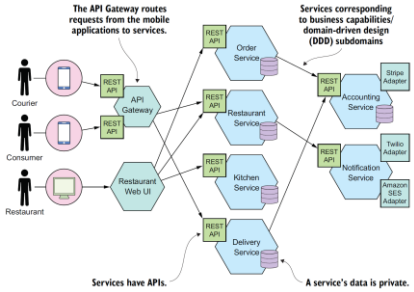
- Interpreters are commonly used to build *virtual machines* that close the gap between the computing engine expected by the semantics of the program and the computing engine available in hardware.



Monolithic Architecture Style



Microservices



Thank You & See You Again